

Q36. UNIQUE PATHS IN GRID

1. Aim

To determine the total number of unique paths from the top-left corner to the bottom-right corner of a grid using **Dynamic Programming**, where the robot can move only **right** or **down**.

2. Problem Statement

A robot is placed at the top-left corner of an $m \times n$ grid. It has to reach the bottom-right corner. At each step, the robot can move only:

- Right $\rightarrow (i, j+1)$
- Down $\rightarrow (i+1, j)$

The objective is to calculate the total number of unique paths efficiently, especially for large grid dimensions.

3. Input Format

m, n = grid dimensions

Sample Input

$m = 3$

$n = 3$

4. Algorithm

1. Create a Dynamic Programming table of size $m \times n$.
2. Initialize the first row and first column to 1, because there is only one way to reach any cell in them.
3. For every remaining cell, calculate:
 $dp[i][j] = dp[i-1][j] + dp[i][j-1]$
4. Continue until the bottom-right cell is reached.

5. The value at `dp[m-1][n-1]` represents the total number of unique paths.
 6. Display the result.
-

5. Pseudocode

UniquePaths(m, n)

Create dp[m][n]

For i = 0 to m-1

 dp[i][0] = 1

For j = 0 to n-1

 dp[0][j] = 1

For i = 1 to m-1

 For j = 1 to n-1

 dp[i][j] = dp[i-1][j] + dp[i][j-1]

Return dp[m-1][n-1]

6. Dynamic Programming Table

For `m = 3` and `n = 3`:

```
+---+---+---+
| 1 | 1 | 1 |
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 1 | 3 | 6 |
+---+---+---+
```

Therefore:

Paths = 6

7. Source Code

```
def unique_paths(m, n):
```

```

# Create DP table
dp = [[0 for _ in range(n)] for _ in range(m)]

# First column
for i in range(m):
    dp[i][0] = 1

# First row
for j in range(n):
    dp[0][j] = 1

# Fill the remaining cells
for i in range(1, m):
    for j in range(1, n):
        dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

return dp[m - 1][n - 1]

# Input
m = 3
n = 3

# Calculate unique paths
paths = unique_paths(m, n)

# Output
print("Grid Dimensions:", m, "x", n)
print("Paths =", paths)

```

8. Sample Input

```

m = 3
n = 3

```

9. Sample Output

Grid Dimensions: 3 x 3

Paths = 6

```
1 def unique_paths(m, n):
2     # First column
3     for i in range(m):
4         dp[i][0] = 1
5
6     # First row
7     for j in range(n):
8         dp[0][j] = 1
9
10    # Fill the remaining cells
11    for i in range(1, m):
12        for j in range(1, n):
13            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
14
15    return dp[m - 1][n - 1]
16
17 # Input
18 m = 3
19 n = 3
20
21 # Calculate unique paths
22 paths = unique_paths(m, n)
23
24 # Output
25 print("Grid Dimensions:", m, "x", n)
26 print("Paths =", paths)
```

Grid Dimensions: 3 x 3
Paths = 6

=== Code Execution Successful ===

10. Explanation

For a 3×3 grid, the robot must make a total of **4 moves**:

- 2 moves Right
- 2 moves Down

The possible arrangements of these moves give a total of:

6 unique paths

Dynamic Programming avoids recalculating the number of paths for previously visited cells, making the solution efficient.

11. Time Complexity

$O(m \times n)$

Each cell in the grid is calculated exactly once.

12. Space Complexity

$O(m \times n)$

A two-dimensional DP table is used to store the number of paths for each cell.

13. Result

Thus, the total number of unique paths from the top-left corner to the bottom-right corner of a 3 × 3 grid was successfully calculated using **Dynamic Programming**.

Result: Paths = 6